

Assignment 2 – Case Study

Stage 4 Tutorial Activities

Object-Oriented Design Decisions

Week 7 | 60 minutes

Activity 1 - Encapsulation Review

Class	Protected state / invariant	Public operations
Patient	Patient name and information. Patient record should be maintained with the correct patient	create_record() search_by_name() update_details() retrieve_information()
Practitioner	Practitioner name and information along with their availability and schedule information should accurately reflect booking information.	view_schedule() view_availability()
Appointment	Appointment date/time, patient, practitioner and status. A practitioner can't have two appointments at the same time and a cancelled appointment can't remain in the history.	book_appointment() cancel_appointment() update_appointment_status()

Activity 2 - Composition or Inheritance?

Appointment and Patient -> ☒ Composition/association ☐ Inheritance Reason: An appointment is associated with a patient, but patients and appointments are different types of objects.

Appointment and Practitioner -> ☒ Composition/association ☐ Inheritance Reason: An appointment is associated with a practitioner who conducted the appointment. These are separate and neither is one another.

Doctor and Practitioner (hypothetical) -> ☐ Composition/association ☒ Inheritance Reason: If the system describes the doctor as a specific type of practitioner, then the doctor would inherit common practitioner traits.

Clinic and Appointment -> ☒ Composition/association ☐ Inheritance Reason: A clinic may have appointments associated with it but an appointment is not a type of clinic.

Activity 3 - Responsibility Allocation

Who decides whether SCHEDULED can become CANCELLED?

The clinic staff that updates the appointment status, as the appointment owns its status.

Who validates a patient name?

Patient should handle specific patient verification. For example, the Patient class can ensure that a name is valid according to whatever validation rules are eventually confirmed.

Should Appointment execute SQL? Why?

No, as the appointment class should represent the appointment and it's business behaviour and not directly execute SQL.

Should the UI decide whether a status transition is legal?

No the appointment class should. The UI might make the request but ultimately the appointment class should determine if the request is granted.

Activity 4 - AI Code Critique

AI generates an Appointment class with public status mutation, SQL inside cancel(), a NotificationManager dependency and inheritance from PatientRecord. Identify at least five design problems and corrections.

Design Problem	Why it's a problem	Correction
Public status mutation	Other classes could directly change status to an invalid value, bypassing appointment business rules.	Keep status controlled by Appointment through methods such as cancel() or update_status().
SQL inside cancel()	Mixes business logic with database/data-access responsibilities, making the class harder to test and maintain.	Keep Appointment responsible for cancellation rules and move SQL/database operations to a separate data-access component.
NotificationManager dependency	SmartCare's confirmed requirements do not specify notifications or reminders. It adds unnecessary coupling and scope.	Remove the dependency unless a future confirmed requirement requires notifications.
Inheritance from PatientRecord	An appointment is not a type of patient record. They represent different domain concepts.	Make Appointment an independent class associated with Patient.
Too many responsibilities in Appointment	SQL, notifications, status management and appointment behaviour violate separation of responsibilities.	Keep Appointment focused on appointment state and business rules such as booking, cancellation and status transitions.

Exit question

Why can code be object-oriented syntactically but still have poor object-oriented design?

Languages like Java and C++ enforce the grammar of certain objects and hence can't enforce how these objects interact or divide responsibilities.